DBMS — LAB 09

Proposed Database & Business Rules Reference

Smart Hostel Management System | CTE and CASE (MySQL)

Important: This is a proposed replacement reference, not the original teacher-issued Lab_9_Data_Contract.pdf. The Student Copy says the original reference defines exact boundaries, tie rules, output columns, sort order, salary revision, and fee rules. Those details were not included in the uploaded materials, so the assumptions below must be confirmed with the instructor before treating the SQL as assessment-ready.

Database: HOSTEL_LAB9 • SQL dialect: MySQL 8.0+ • Permitted techniques: joins, aggregates, subqueries, CTEs, recursive CTEs, CASE

1. Source facts and limitations

The uploaded Student Copy identifies the tables HOSTEL, STAFF, ROOM, BED, STUDENT, STUDENT_PHONE, FACILITY, USES, ALLOCATE, PAYMENT, and RECEIPT. It states that current residents are associated with active allocations, that staff can have a self-referencing supervisor hierarchy, and that room occupancy must be reconciled against bed availability flags. It does not state the exact business-rule constants needed for all six questions.

The Task 1 dataset provides the schema and supplementary records, but the supplied materials do not include the separate Lab 9 Data Contract. Therefore this document distinguishes source-backed schema facts from proposed rules.

2. Proposed conventions (must be confirmed)

Area	Proposed interpretation
Current resident	A student with an ALLOCATE row where Check_out_Date IS NULL. If Check_in_Date is present, require Check_in_Date <= CURRENT_DATE; for reproducible lab outputs, prefer a fixed reference date supplied by the instructor.
Resident payment total	Sum PAYMENT.Amount for a student. A student with no payments has total 0. Hostel resident totals are calculated for current residents only.
Campus benchmark	Average of each hostel's average current-resident payment total, including hostels with zero current residents; ties at the maximum payer amount are preserved.
Superior salary inversion	A staff member has an inversion against each ancestor in the STAFF.Supervisor_ID chain whose salary is lower than the staff member's salary. Include all ancestor levels, not only the direct supervisor.
Room occupancy reconciliation	For each room, compare active resident count with BED rows marked Availability='N'. Proposed statuses: MATCH, TOO_MANY_OCCUPIED_FLAGS, TOO_FEW_OCCUPIED_FLAGS, NO_BEDS. Proposed correction count is the absolute difference between occupied flags and current residents; NO_BEDS with residents is treated as a mismatch.
Salary revision	No salary increase percentages or cap formula appear in the supplied Student Copy. These must be supplied by the instructor; do not infer them from the dataset.
Facility engagement	Count distinct facilities recorded in USES for each current resident's home hostel. Proposed classes: NONE (0), LOW (1), MODERATE (2), HIGH (3 or more). Report every class tied for the highest resident count in each hostel.
Daily fee ledger	No daily base fee, due-date rule, grace period, daily late-fee rate, compounding method, payment allocation order, or end date appears in the supplied materials. These must be supplied by the instructor; do not infer them.

3. Question-by-question contract outline

C1 — Resident payment leaders

Compute each current resident's total payment amount, then each hostel's average resident total. Compare each hostel average to the campus benchmark. Return qualifying hostels and all residents tied for that hostel's maximum payment total. Proposed tie policy: preserve ties. Exact benchmark definition needs instructor confirmation.

C2 — Superior salary inversions

Traverse STAFF.Supervisor_ID recursively from each staff member to all ancestors. Return staff members whose salary exceeds one or more ancestor salaries and summarize the number of inversions and/or salary differences. Exact output columns and ordering are not provided.

C3 — Room occupancy reconciliation

For every ROOM row, count bed rows and occupied flags (Availability='N'), and count current allocations. Assign exactly one status using the proposed convention in section 2, then aggregate status counts and correction totals by hostel. Exact status names may differ from the original contract.

C4 — Staff salary revision

Apply the published salary adjustment to every staff member and enforce supervisor caps before calculating current and revised payroll by hostel. The adjustment rates, cap definition, whether the cap is against direct supervisor or all ancestors, and rounding policy are missing. This question cannot be completed reliably without these values.

C5 — Facility engagement profile

For each current resident, count facilities linked through USES and FACILITY to their home hostel. Assign one engagement class and report the most common class or classes per hostel. The class boundaries and treatment of facilities outside the home hostel are assumptions, not source facts.

C6 — Daily fee ledger

Generate a date sequence for each resident, calculate daily fee accrual, apply late-fee compounding, and report penalized students' fees and ending balance. Base fee, date range, grace period, late rate, compounding order, rounding, and payment offsets are missing; no exact answer should be claimed without them.

4. MySQL and submission constraints

- Each answer must be one read-only MySQL statement.
- Use CTEs and WITH RECURSIVE where needed; use CASE for conditional classification.
- Do not use window functions, user variables, stored programs, views, or temporary tables.
- Do not modify the dataset.
- The required student deliverables are an SQL file with six labeled queries and a PDF showing each query with complete output in the MySQL command-line client.
- Do not fabricate terminal screenshots or label unexecuted SQL as verified output.

Recommended next step: ask the instructor to confirm the missing C1–C6 constants and output specifications. This proposed reference is useful for organizing the work, but it is not a substitute for the official assessment contract.